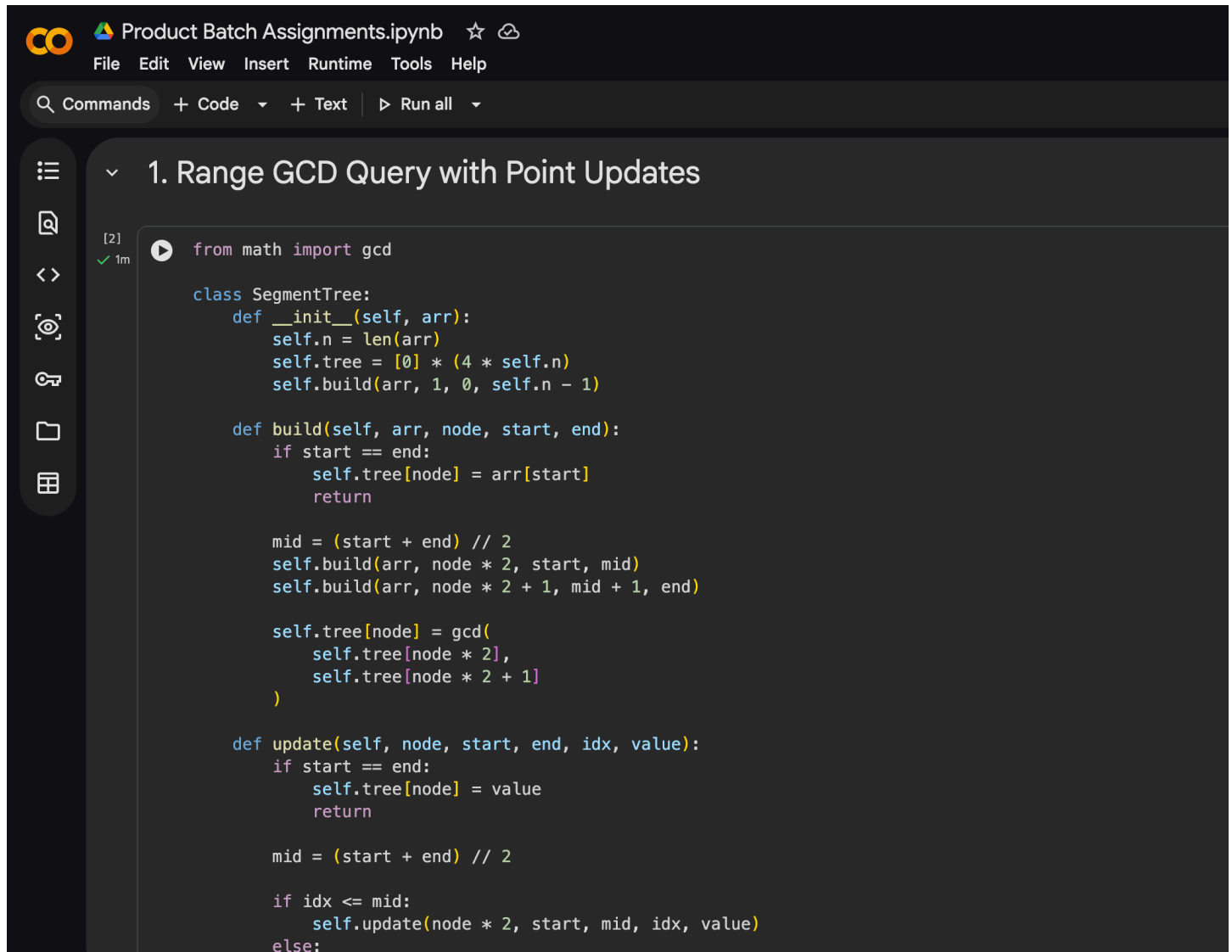


# Product Batch - Task Submission - 02.09.2026

ROLL NO : 2303A51027

NAME : Aravind Reddy

## 1. Range GCD Query with Point Updates



The screenshot displays a Jupyter Notebook titled "Product Batch Assignments.ipynb". The notebook is open to a cell containing Python code for a Segment Tree implementation to handle Range GCD Queries and Point Updates. The code defines a `SegmentTree` class with methods for building the tree and updating values. The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a search bar, and a left sidebar with navigation icons. The code cell is labeled [2] and shows a successful execution status (green checkmark) with a runtime of 1m.

```
[2] ✓ 1m
from math import gcd

class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self.build(arr, 1, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
            return

        mid = (start + end) // 2
        self.build(arr, node * 2, start, mid)
        self.build(arr, node * 2 + 1, mid + 1, end)

        self.tree[node] = gcd(
            self.tree[node * 2],
            self.tree[node * 2 + 1]
        )

    def update(self, node, start, end, idx, value):
        if start == end:
            self.tree[node] = value
            return

        mid = (start + end) // 2

        if idx <= mid:
            self.update(node * 2, start, mid, idx, value)
        else:
```



Q Commands + Code + Text ▶ Run all

[2]  
✓ 1m

```
self.tree[node] = gcd(  
    self.tree[node * 2],  
    self.tree[node * 2 + 1]  
)
```

```
def query(self, node, start, end, l, r):  
    if r < start or end < l:  
        return 0  
  
    if l <= start and end <= r:  
        return self.tree[node]  
  
    mid = (start + end) // 2  
  
    left = self.query(node * 2, start, mid, l, r)  
    right = self.query(node * 2 + 1, mid + 1, end, l, r)  
  
    return gcd(left, right)
```

```
n = int(input())  
arr = list(map(int, input().split()))  
  
st = SegmentTree(arr)  
  
q = int(input())  
  
for _ in range(q):  
    data = list(map(int, input().split()))  
  
    if data[0] == 1:  
        idx, value = data[1], data[2]  
        st.update(1, 0, n - 1, idx, value)  
    else:  
        l, r = data[1], data[2]  
        print(st.query(1, 0, n - 1, l, r))
```

CO

Product Batch Assignments.ipynb

☆

🔗

FileEditViewInsertRuntimeToolsHelp

🔍 Commands+ Code+ Text▶ Run all

☰

🔍

⏪

🔍

🔑

📁

📅

[2]  
✓ 1m

▶

```
        if l <= start and end <= r:
            return self.tree[node]

        mid = (start + end) // 2

        left = self.query(node * 2, start, mid, l, r)
        right = self.query(node * 2 + 1, mid + 1, end, l, r)

        return gcd(left, right)

n = int(input())
arr = list(map(int, input().split()))

st = SegmentTree(arr)

q = int(input())

for _ in range(q):
    data = list(map(int, input().split()))

    if data[0] == 1:
        idx, value = data[1], data[2]
        st.update(1, 0, n - 1, idx, value)
    else:
        l, r = data[1], data[2]
        print(st.query(1, 0, n - 1, l, r))
```

▼

...

4

1 3 2 4

3

1 3 2

1 2 3

3 2 1

0

## 2. Range Product Query



## 2. Range Product Query

[4]  
✓ 23s

MOD = 1000000007

```
class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [1] * (4 * self.n)
        self.build(arr, 1, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start] % MOD
            return

        mid = (start + end) // 2

        self.build(arr, node * 2, start, mid)
        self.build(arr, node * 2 + 1, mid + 1, end)

        self.tree[node] = (
            self.tree[node * 2] *
            self.tree[node * 2 + 1]
        ) % MOD

    def update(self, node, start, end, idx, value):
        if start == end:
            self.tree[node] = value % MOD
            return

        mid = (start + end) // 2

        if idx <= mid:
            self.update(node * 2, start, mid, idx, value)
```



Q Commands + Code + Text ▶ Run all



[4]

✓ 23s



```
        if idx <= mid:
            self.update(node * 2, start, mid, idx, value)
        else:
            self.update(node * 2 + 1, mid + 1, end, idx, value)

        self.tree[node] = (
            self.tree[node * 2] *
            self.tree[node * 2 + 1]
        ) % MOD

    def query(self, node, start, end, l, r):
        if r < start or end < l:
            return 1

        if l <= start and end <= r:
            return self.tree[node]

        mid = (start + end) // 2

        left = self.query(node * 2, start, mid, l, r)
        right = self.query(node * 2 + 1, mid + 1, end, l, r)

        return (left * right) % MOD

n = int(input())
arr = list(map(int, input().split()))

st = SegmentTree(arr)

q = int(input())

for _ in range(q):
    data = list(map(int, input().split()))

    if data[0] == 1:
        st.update(1, 0, n - 1, data[1], data[2])
```

CO Product Batch Assignments.ipynb ☆ ☁  
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

☰

🔍

<>

🔍

🔑

📁

📅

[4] ✓ 23s ▶

```
def query(self, node, start, end, l, r):  
    if r < start or end < l:  
        return 1  
  
    if l <= start and end <= r:  
        return self.tree[node]  
  
    mid = (start + end) // 2  
  
    left = self.query(node * 2, start, mid, l, r)  
    right = self.query(node * 2 + 1, mid + 1, end, l, r)  
  
    return (left * right) % MOD  
  
n = int(input())  
arr = list(map(int, input().split()))  
  
st = SegmentTree(arr)  
  
q = int(input())  
  
for _ in range(q):  
    data = list(map(int, input().split()))  
  
    if data[0] == 1:  
        st.update(1, 0, n - 1, data[1], data[2])  
    else:  
        print(st.query(1, 0, n - 1, data[1], data[2]))
```

▼ ... 4  
1 3 2 4  
2  
1 2 4  
2 3 1  
1

### 3. Range Bitwise OR Query



## ▼ 3. Range Bitwise OR Query

[5]  
✓ 20s

```
class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self.build(arr, 1, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
            return

        mid = (start + end) // 2

        self.build(arr, node * 2, start, mid)
        self.build(arr, node * 2 + 1, mid + 1, end)

        self.tree[node] = (
            self.tree[node * 2] |
            self.tree[node * 2 + 1]
        )

    def update(self, node, start, end, idx, value):
        if start == end:
            self.tree[node] = value
            return

        mid = (start + end) // 2

        if idx <= mid:
            self.update(node * 2, start, mid, idx, value)
        else:
            self.update(node * 2 + 1, mid + 1, end, idx, value)
```



Q Commands + Code + Text ▶ Run all

[5]  
✓ 20s

```
self.tree[node] = (  
    self.tree[node * 2] |  
    self.tree[node * 2 + 1]  
)  
  
def query(self, node, start, end, l, r):  
    if r < start or end < l:  
        return 0  
  
    if l <= start and end <= r:  
        return self.tree[node]  
  
    mid = (start + end) // 2  
  
    left = self.query(node * 2, start, mid, l, r)  
    right = self.query(node * 2 + 1, mid + 1, end, l, r)  
  
    return left | right  
  
n = int(input())  
arr = list(map(int, input().split()))  
  
st = SegmentTree(arr)  
  
q = int(input())  
  
for _ in range(q):  
    data = list(map(int, input().split()))  
  
    if data[0] == 1:  
        st.update(1, 0, n - 1, data[1], data[2])  
    else:  
        print(st.query(1, 0, n - 1, data[1], data[2]))
```

CO

Product Batch Assignments.ipynb

☆

🔗

File Edit View Insert Runtime Tools Help

🔍 Commands + Code + Text ▶ Run all

☰

🔍

⏮

🔍

🔑

📁

📅

[5] ✓ 20s ▶

```
def query(self, node, start, end, l, r):
    if r < start or end < l:
        return 0

    if l <= start and end <= r:
        return self.tree[node]

    mid = (start + end) // 2

    left = self.query(node * 2, start, mid, l, r)
    right = self.query(node * 2 + 1, mid + 1, end, l, r)

    return left | right

n = int(input())
arr = list(map(int, input().split()))

st = SegmentTree(arr)

q = int(input())

for _ in range(q):
    data = list(map(int, input().split()))

    if data[0] == 1:
        st.update(1, 0, n - 1, data[1], data[2])
    else:
        print(st.query(1, 0, n - 1, data[1], data[2]))
```

⌵

... 4  
1 3 2 4  
2  
1 3 2  
2 1 3  
3

#### 4. K-th One in a Binary Array



Commands + Code + Text Run all

[6]  
✓ 22s

## 4. K-th One in a Binary Array

```
class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self.build(arr, 1, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
            return

        mid = (start + end) // 2

        self.build(arr, node * 2, start, mid)
        self.build(arr, node * 2 + 1, mid + 1, end)

        self.tree[node] = (
            self.tree[node * 2] +
            self.tree[node * 2 + 1]
        )

    def update(self, node, start, end, idx, value):
        if start == end:
            self.tree[node] = value
            return

        mid = (start + end) // 2

        if idx <= mid:
            self.update(node * 2, start, mid, idx, value)
        else:
            self.update(node * 2 + 1, mid + 1, end, idx, value)
```

{6}  
✓ 22s

```
self.tree[node] = (  
    self.tree[node * 2] +  
    self.tree[node * 2 + 1]  
)  
  
def kth_one(self, node, start, end, k):  
    if start == end:  
        return start  
  
    mid = (start + end) // 2  
  
    left_ones = self.tree[node * 2]  
  
    if k <= left_ones:  
        return self.kth_one(node * 2, start, mid, k)  
  
    return self.kth_one(  
        node * 2 + 1,  
        mid + 1,  
        end,  
        k - left_ones  
    )  
  
n = int(input())  
arr = list(map(int, input().split()))  
  
st = SegmentTree(arr)  
  
q = int(input())  
  
for _ in range(q):  
    data = list(map(int, input().split()))  
  
    if data[0] == 1:  
        st.update(1, 0, n - 1, data[1], data[2])  
    else:
```

[6]  
✓ 22s

```
        if k <= left_ones:
            return self.kth_one(node * 2, start, mid, k)

        return self.kth_one(
            node * 2 + 1,
            mid + 1,
            end,
            k - left_ones
        )

n = int(input())
arr = list(map(int, input().split()))

st = SegmentTree(arr)

q = int(input())

for _ in range(q):
    data = list(map(int, input().split()))

    if data[0] == 1:
        st.update(1, 0, n - 1, data[1], data[2])
    else:
        k = data[1]

        if st.tree[1] < k:
            print(-1)
        else:
            print(st.kth_one(1, 0, n - 1, k))
```

▼

```
... 4
    1 2 4 3
    2
    1 4 2
    2 5 3
    2
```



## 5. First Element Greater Than or Equal to X

[7]  
✓ 22s

```
class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [float('-inf')] * (4 * self.n)
        self.build(arr, 1, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
            return

        mid = (start + end) // 2

        self.build(arr, node * 2, start, mid)
        self.build(arr, node * 2 + 1, mid + 1, end)

        self.tree[node] = max(
            self.tree[node * 2],
            self.tree[node * 2 + 1]
        )

    def update(self, node, start, end, idx, value):
        if start == end:
            self.tree[node] = value
            return

        mid = (start + end) // 2

        if idx <= mid:
            self.update(node * 2, start, mid, idx, value)
        else:
            self.update(node * 2 + 1, mid + 1, end, idx, value)
```

## 5. First Element Greater Than or Equal to X



## 5. First Element Greater Than or Equal to X

[7]  
✓ 22s

```
class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [float('-inf')] * (4 * self.n)
        self.build(arr, 1, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
            return

        mid = (start + end) // 2

        self.build(arr, node * 2, start, mid)
        self.build(arr, node * 2 + 1, mid + 1, end)

        self.tree[node] = max(
            self.tree[node * 2],
            self.tree[node * 2 + 1]
        )

    def update(self, node, start, end, idx, value):
        if start == end:
            self.tree[node] = value
            return

        mid = (start + end) // 2

        if idx <= mid:
            self.update(node * 2, start, mid, idx, value)
        else:
            self.update(node * 2 + 1, mid + 1, end, idx, value)
```



Q Commands + Code + Text ▶ Run all

[7]  
✓ 22s

```
        self.tree[node] = max(
            self.tree[node * 2],
            self.tree[node * 2 + 1]
        )

    def find_first(self, node, start, end, l, r, x):

        if end < l or start > r:
            return -1

        if self.tree[node] < x:
            return -1

        if start == end:
            return start

        mid = (start + end) // 2

        # Search left first
        ans = self.find_first(
            node * 2, start, mid, l, r, x
        )

        if ans != -1:
            return ans

        return self.find_first(
            node * 2 + 1, mid + 1, end, l, r, x
        )

n = int(input())
arr = list(map(int, input().split()))

st = SegmentTree(arr)
```



Q Commands + Code + Text ▶ Run all ▼

[7]  
✓ 22s

```
▶ n = int(input())
  arr = list(map(int, input().split()))

  st = SegmentTree(arr)

  q = int(input())

  for _ in range(q):
      data = list(map(int, input().split()))

      if data[0] == 1:
          st.update(1, 0, n - 1, data[1], data[2])
      else:
          l, r, x = data[1], data[2], data[3]

          print(st.find_first(
              1, 0, n - 1, l, r, x
          ))
```

▼ ... 4  
1 3 4 5  
2  
1 4 2 5  
1 3 4 6

[ ]

▶ Start coding or [generate](#) with AI.